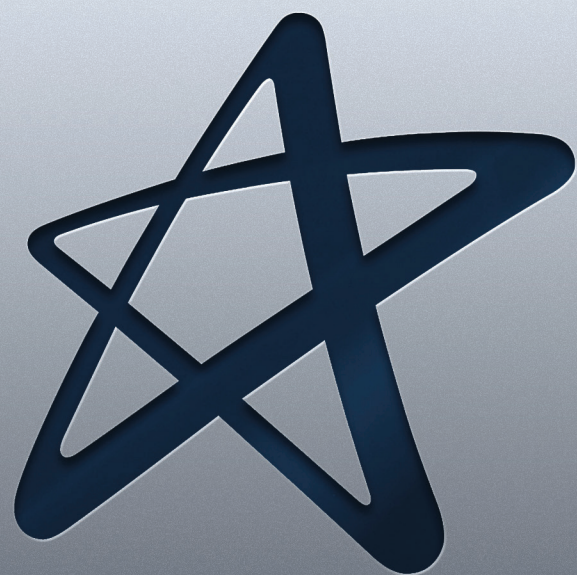


Estruturas de Dados Lineares



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Algoritmos de Classificação e Pesquisa II

Responsável pelo Conteúdo:

Prof. Dr. Luciano Rossi

Revisão Textual:

Prof.^a Esp. Kelciane da Rocha Campos

UNIDADE

Algoritmos de Classificação e Pesquisa II



- Introdução;
- Recursividade;
- Ordenação pelo Método Rápido (*Quick Sort*);
- Ordenação por Mistura (*Merge Sort*);
- Pesquisa Binária.



OBJETIVOS DE APRENDIZADO

- Apresentar ao aluno o conceito de recursividade no contexto da computação;
- Estudar os principais algoritmos de ordenação que utilizam a abordagem recursiva;
- Identificar a diferença de desempenho da pesquisa binária considerando, também, a abordagem recursiva.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e sites para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Na unidade anterior, tivemos a oportunidade de estudar alguns dos principais algoritmos de ordenação. Esses algoritmos têm uma característica em comum: além do objetivo, todos eles utilizam uma estratégia iterativa em sua lógica para a ordenação. A iteração, no contexto da programação de computadores, se refere à repetição de uma ou mais instruções.

Além disso, vimos como a pesquisa ou busca por um determinado elemento pode ser feita em um vetor (arranjo), considerando também uma abordagem iterativa.

Nesta nova unidade, daremos sequência ao estudo dos algoritmos de ordenação e de busca. A diferença fundamental para estes novos algoritmos é sobre a utilização de uma abordagem recursiva na estruturação das soluções. A recursão é uma abordagem na qual a execução de um determinado procedimento prevê uma nova execução do mesmo procedimento. Um exemplo lúdico de recursividade é quando colocamos dois espelhos, um de frente ao outro, de modo que um reflita o outro. A imagem no espelho refletirá o outro espelho que, por sua vez, está refletindo a imagem do primeiro e assim sucessiva e indefinidamente.



Você poderia identificar outros exemplos de recursividade que podem ser observados no nosso cotidiano?

Os algoritmos recursivos possuem uma chamada interna a si mesmo. Assim, uma nova execução do algoritmo se inicia, ficando a primeira execução aguardando a finalização da segunda. Esse processo pode ser repetido diversas vezes até que um determinado caso base ocorra.

A recursividade é utilizada em algoritmos que utilizam uma estratégia denominada divisão e conquista. Essa estratégia consiste em dividir um problema, sucessivas vezes, até que ele seja pequeno o suficiente e possa ser resolvido de forma direta.

Na matemática, há muita aplicação do conceito de recursão para a definição de, por exemplo, conjuntos de objetos. O conjunto dos números naturais ($\mathbb{N}$), por exemplo, pode ser definido recursivamente. Dizemos que o zero é um número natural ($0 \in \mathbb{N}$) e se um número n é natural ($n \in \mathbb{N}$), então $n + 1$ também é ($n + 1 \in \mathbb{N}$).

Veremos nas próximas seções como utilizar a recursividade em nossos algoritmos, principalmente para aqueles que utilizam a abordagem da divisão e conquista. Em particular, estudaremos os algoritmos de ordenação pelo método rápido (*Quick Sort*), por mistura (*Merge Sort*) e a pesquisa binária. Iniciaremos com uma descrição detalhada sobre a recursividade.

Recursividade

Na programação, a recursividade é a capacidade que uma função apresenta de chamar a si mesma. A ideia fundamental por trás de um algoritmo recursivo é a transformação do problema (instância) original em outro menor ou mais simples, de modo que seu

tamanho ou sua simplicidade permita uma solução direta, sem que haja a necessidade de recorrer novamente ao algoritmo.

A condição de parada de um algoritmo recursivo é denominada de caso base. Quando, no processo recursivo, atinge-se o caso base, o algoritmo interrompe as chamadas recursivas e retorna a solução direta, para esse caso, à instância que realizou a chamada, até que a solução para o problema original seja obtida.

Todo algoritmo recursivo terá uma versão iterativa para executar a mesma tarefa. Quando comparamos as duas abordagens, observamos que os algoritmos recursivos são mais simples de compreender e apresentam um número menor de instruções. Além disso, existem tipos de problema que apresentam características recursivas, o que sugere uma solução na mesma abordagem.

As desvantagens de se utilizar um algoritmo recursivo estão ligadas ao consumo de recursos. É muito comum que instâncias muito grandes acabem por consumir muita memória, por conta do empilhamento de cada chamada realizada. Além disso, os algoritmos recursivos são mais difíceis de serem depurados.

Além da computação, a matemática é uma área que utiliza muito a recursividade. Um exemplo clássico é a definição do fatorial. O fatorial de um número natural é o produto do próprio número por todos os seus antecessores não nulos. Assim, podemos dar uma definição formal, utilizando recursão, da seguinte maneira:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n(n-1)! & \text{se } n > 0. \end{cases}$$

Veja que o fatorial de zero é igual a um ($0! = 1$), esse é o caso base. Quando o valor de n for zero, resolvemos de forma direta. Para os demais casos, quando n for maior que zero, o tamanho da instância é reduzido ($n - 1$) e aplicamos a mesma definição para essa nova instância.

Por exemplo, suponha $n = 3$. Nesse caso, teríamos $3(2(1(0!)))$. Veja que o valor de n foi reduzido até chegar a zero, que é o caso base. A partir do caso base, para o qual conhecemos o resultado, podemos calcular as demais instâncias $3(2(1(1))) = 6$.

Algoritmo 1 – Algoritmo recursivo para o cálculo do fatorial de um número

Um algoritmo recursivo, para o cálculo do fatorial de um número, poderia ter suas instruções descritas como no Algoritmo 1. Veja que o condicional na linha 1 verifica o caso base ($n = 0$) e a estratégia de resolução e a redução do tamanho da instância ($n * (n - 1)!$) são descritas na linha 4.

Fatorial(n)

1. **se** $n = 0$
2. **retornar** 1
3. **senão**
4. **retornar** $n * \text{Fatorial}(n-1)$

Os algoritmos que consideram a abordagem recursiva sempre descrevem, pelo menos, um caso base, a redução do tamanho do problema e uma nova chamada a si mesmo. Outro exemplo clássico de recursividade é o cálculo do valor da n -ésima posição da sequência de Fibonacci.



Descrição detalhada da série de Fibonacci e possíveis aplicações: <https://bit.ly/3r1vNRC>

A sequência, ou série, de Fibonacci tem os valores iniciais 0 e 1, o que se caracteriza como o caso base da recursão. Os demais valores são obtidos a partir da soma dos dois anteriores, essa é a regra de formação. A recursividade, para esse caso, pode ser evidenciada na representação formal do problema, a qual define uma função que recebe a posição desejada e retorna o valor da posição.

$$Fib(n) = \begin{cases} 0 & \text{sen } n = 1 \\ 1 & \text{sen } n = 2 \\ Fib(n-1) + Fib(n-2) & \text{sen } n > 2. \end{cases}$$

Veja que quando $n = 1$ se espera, como resultado da função $Fib(n)$, o valor da primeira posição, que, no caso, é igual a 0. Nesse sentido, quando $n = 2$ queremos o valor da segunda posição, que é igual a 1. Esses casos são os casos base; a partir daí, há a redução do tamanho da instância e duas chamadas recursivas são feitas à função $Fib()$.

Cada chamada gera um novo “ramo” de computação, que é resultado da nova chamada à função. Nesse contexto, a execução que contém a chamada fica aguardando a finalização da execução de todas as chamadas que foram derivadas dela. Assim, a execução chamadora será finalizada a partir dos resultados retornados pelas chamadas derivadas. A Figura 2 apresenta uma representação dessas chamadas com os respectivos retornos e finalização.

A Figura 2 apresenta a árvore de chamadas à função $Fib()$ com suas respectivas relações. A chamada inicial é $Fib(5)$; como esse não é um dos casos base, duas novas chamadas recursivas são feitas, $Fib(4)$ e $Fib(3)$. Ambas as chamadas recursivas, também, não correspondem aos casos base e realizam novas chamadas. Veja que três dessas novas chamadas correspondem aos casos base e, assim, elas retornam os seus respectivos valores. Os casos base estão destacados na cor vermelha na árvore e os valores de retorno estão em verde.

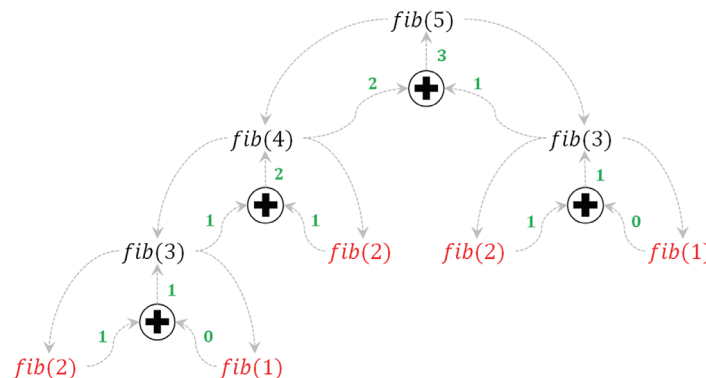


Figura 1 – Chamadas recursivas do algoritmo de Fibonacci

Fonte: Acervo do conteudista

Quando ambas as chamadas recursivas são casos base, os valores de retorno são somados e passados para a função chamadora. Esse processo vai se repetir até que todos os ramos atinjam os casos base e retornem seus valores até a função inicial. Veja que, para esse caso, a função *Fib* (5) resultará no valor 3, que é o quinto elemento da sequência.

Uma característica importante dos algoritmos recursivos é que eles podem realizar várias chamadas para uma mesma instância. Veja que, para o exemplo da Figura 3, a chamada *Fib* (3) é realizada duas vezes. Isso significa que essa instância foi computada duas vezes, o que é desnecessário e acarretará um maior tempo de processamento.

Algoritmo 2 – Algoritmo recursivo para o cálculo do *n*-ésimo valor da série de Fibonacci

O algoritmo recursivo que retorna o valor da *n*-ésima posição da série de Fibonacci é descrito no Algoritmo 2. Os casos base são testados na linha 1 e, caso se confirmem, será retornada a posição menos uma unidade, o que corresponde ao valor da posição na série. Nesse contexto, os demais casos geram as duas novas chamadas recursivas (linha 4), cada uma com uma considerando uma instância reduzida.

Fib(*n*)

1. **se** *n* = 1 ou *n* = 2
2. **retornar** *n* – 1
3. **senão**
4. **retornar** *Fib*(*n* – 1) + *Fib*(*n* – 2)

Os algoritmos recursivos são, de fato, muito úteis para a computação, porém seu uso deve ser considerado com cuidado. Como descrito anteriormente, quando temos instâncias grandes as chamadas que ficam empilhadas na memória podem consumir muito desse recurso, tornando o processamento inviável.



Para todo algoritmo recursivo existe um correspondente iterativo que realiza a mesma tarefa. Como seria a versão iterativa do algoritmo *Fib* ()?

Na próxima seção, discutiremos o algoritmo de ordenação pelo método rápido, também conhecido como *Quick Sort*. Esse algoritmo considera a abordagem recursiva para ordenar um vetor. A estratégia do *Quick Sort* resulta em um desempenho, em termos de tempo de execução, muito melhor do que os desempenhos dos algoritmos de ordenação estudados na unidade anterior.

Ordenação pelo Método Rápido (*Quick Sort*)

O algoritmo de ordenação pelo método rápido, conhecido como *Quick Sort*, foi desenvolvido pelo cientista da computação Charles Antony Richard Hoare em 1960. Hoare é responsável por diversas contribuições importantes no campo da Ciência da Computação e foi homenageado com diferentes prêmios importantes para a área, inclusive o Prêmio Turing em 1980.

O *Quick Sort* é, possivelmente, o algoritmo de ordenação mais utilizado no mundo e apresenta um desempenho muito bom em termos de tempo de execução. Esse algoritmo utiliza o paradigma da divisão e conquista para ordenar um vetor típico $V = [v_p, v_{p+1}, \dots, v_{r-1}, v_r]$.

A implementação do *Quick Sort* considera dois procedimentos. O primeiro procedimento, aqui denominado *Quicksort()*, reorganiza o vetor V em dois subvetores, $V' = [v_p, v_{p+1}, \dots, v_{q-2}, v_{q-1}]$ e $V'' = [v_{q+1}, v_{q+2}, \dots, v_{r-1}, v_r]$, de modo que $v_i \in V' \leq v_q \leq v_j \in V''$. O segundo procedimento, denominado *Partition()*, é a chave para que o procedimento *Quicksort()* possa reorganizar o vetor.

Algoritmo 3 – Algoritmo *Quick Sort*

Quicksort(V, p, r)

1. **se** $p < q$
2. $q \leftarrow \text{Partition}(V, p, r)$
3. *Quicksort*($V, p, q - 1$)
4. *Quicksort*($V, q + 1, r$)

O Algoritmo 3 descreve as instruções que compõem o procedimento *Quicksort()*. O algoritmo recebe como parâmetros de entrada o vetor V , que será ordenado; o inteiro p , que é o indexador da primeira posição do vetor; e o inteiro r que indica a última posição do vetor, de modo que $r = |V| - 1$.

Inicialmente, o algoritmo verifica se o vetor não é unitário; veja que se os indexadores p e r forem iguais, isso indica que existe apenas um elemento no vetor e, dessa forma, ele já se encontra ordenado. O próximo passo é identificar a posição no vetor que será utilizada para desmembrá-lo em dois subvetores, de modo que os valores no primeiro subvetor sejam menores ou iguais ao valor da posição utilizada e, da mesma forma, os valores no segundo subvetor devem ser maiores ou iguais ao valor de posição que os separa.

Após a divisão, cada subvetor obtido será submetido, recursivamente, ao procedimento *Quicksort()*. Como descrito anteriormente, a chave para essa divisão está descrita pelo procedimento *Partition()*, cujas instruções são apresentadas no Algoritmo 4.

Algoritmo 4 – Procedimento *Partition*

$Partition(V, p, r)$

1. $x \leftarrow V[r]$
2. $i \leftarrow p - 1$
3. **para** $j \leftarrow p$ **até** $r - 1$
4. **se** $V[j] \leq x$
5. $i \leftarrow i + 1$
6. **trocar** $V[i]$ e $V[j]$
7. **trocar** $V[i + 1]$ e $V[r]$
8. **retornar** $i + 1$

O procedimento *Partition()* recebe como parâmetros de entrada o vetor V e os indexadores p e r , que representam a primeira e a última posições no vetor, respectivamente. O valor da última posição no vetor é reservado por meio de sua atribuição à variável x , na linha 1.

A variável i é um ponteiro que, inicialmente, estará indicando uma posição fora do vetor, imediatamente anterior à primeira posição, conforme indicado na linha 2. O laço de repetição, na linha 3, gerará os valores dos indexadores desde a primeira posição no vetor até a penúltima. Assim, o vetor será percorrido de acordo com os indexadores gerados pelo laço. Caso o valor armazenado na posição apontada por j , indicada pelo percurso do laço, seja menor ou igual ao valor reservado na variável x , o ponteiro i é incrementado em uma unidade e ocorrerá a troca entre os valores na posição apontada pelo ponteiro i e a posição corrente apontada por j , que é definida pelo laço.

Finalmente, ocorre a troca entre os valores nas posições $i + 1$ e r . O procedimento retornará o valor $i + 1$, que representa a posição que será considerada para a divisão do vetor pelo algoritmo *Quicksort()*. A Figura 2 ilustra a execução do procedimento *Partition()* sobre um vetor exemplo.

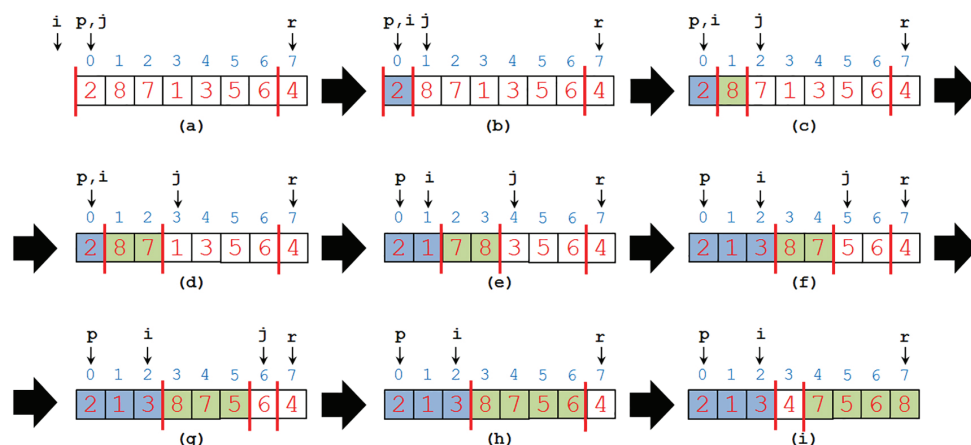


Figura 2 – Descrição da execução do procedimento *Partition* sobre um vetor de inteiros

Fonte: Acervo do conteudista

A Figura 2 descreve os passos de acordo com as instruções do procedimento *Partition()*. Veja que, inicialmente (passo a), os ponteiros estão organizados de modo que possibilitem a execução do percurso do vetor, da primeira até a penúltima posição. Os valores, apontados por j , são comparados com o valor apontado por r , e quando o primeiro é menor ou igual ao segundo, o ponteiro i avança uma posição e os valores apontados por i e j são trocados entre si.

Veja que o ponteiro i avança de modo a separar os valores que são menores que aquele apontado por r . No passo (f), por exemplo, os valores à esquerda de i , destacados na cor azul, são menores que o valor apontado por r e os valores alcançados por j , que são destacados na cor verde, são maiores que o valor apontado por r . Desse modo, quando o percurso de j terminar, as posições $i + 1$ e r têm seus valores trocados no passo (i).

No passo i, temos o estado final do vetor após a execução do procedimento *Partition()*. O valor retornado pelo procedimento à função chamadora *Quicksort()* será, nesse exemplo, igual a três, o que corresponde a $i + 1$. Veja que todos os valores destacados em azul são menores que o valor da posição indicada por r e os valores em verde são maiores. Assim, esses subvetores (azul e verde) serão enviados, recursivamente, à função *Quicksort()*, que repetirá o procedimento *Partition()* até que o vetor esteja ordenado.

O algoritmo de ordenação pelo método rápido é, muitas vezes, a melhor opção para ordenação de arranjos. Isso se deve ao seu ótimo desempenho médio, fazendo com que o *Quick Sort* seja o algoritmo mais considerado entre aqueles pertencentes à classe dos algoritmos de ordenação.

Ordenação por Mistura (*Merge Sort*)

O algoritmo de ordenação por mistura ou intercalação, mais conhecido como *Merge Sort*, também considera a abordagem da divisão e conquista, assim como o seu correlato *Quick Sort*. O princípio fundamental do *Merge Sort* está relacionado com a combinação de dois subvetores ordenados em um único vetor ordenado.

Algoritmo 5 – Algoritmo de ordenação pelo método de intercalação (*Merge Sort*), esse procedimento é parte do algoritmo completo

O Algoritmo 5 descreve as instruções para o procedimento *Merge()*, que realiza a combinação de vetores previamente ordenados. Esse procedimento considera como parâmetros de entrada um vetor V , no qual os valores à esquerda da posição q estão ordenados e os valores à direita de q também estão ordenados. As variáveis p e r contêm as posições inicial e final do vetor, respectivamente.

O procedimento cria, inicialmente, dois subvetores, E e D , que serão responsáveis por armazenar os subvetores esquerdo e direito, derivados do vetor V (linhas 1-7). Os subvetores têm cada um uma posição adicional, que será ocupada por um valor sentinela. Para esse exemplo, o valor sentinela utilizado é o ∞ (infinito). O papel da sentinela é marcar o final do subvetor.

A combinação dos subvetores é feita a partir da seleção do menor valor entre aqueles que estão em suas posições iniciais, de modo que sempre o menor valor será inserido

primeiro no vetor. Veja que, como os subvetores estão ordenados, um valor nas posições iniciais dos subvetores é sempre menor ou igual aos valores subsequentes.

Veja que as variáveis i e j marcam as primeiras posições de cada subvetor. Assim, quando o valor na posição inicial de um dos subvetores é selecionado, a variável correspondente a ele é incrementada, de modo a apontar para um novo valor inicial.

Merge(V, p, q, r)

1. $n_1 \leftarrow q - p + 1$
2. $n_2 \leftarrow r - q$
3. *sejam* $E[0 \dots n_1]$ e $D[0 \dots n_2]$ *novos vetores*
4. **para** $i \leftarrow 0$ **até** $n_1 - 1$
5. $E[i] \leftarrow V[p + i]$
6. **para** $j \leftarrow 0$ **até** $n_2 - 1$
7. $D[j] \leftarrow V[q + j + 1]$
8. $E[n_1] \leftarrow \infty$
9. $D[n_2] \leftarrow \infty$
10. $i \leftarrow 0$
11. $j \leftarrow 0$
12. **para** $k \leftarrow p$ **até** r
13. **se** $E[i] \leq D[j]$
14. $V[k] \leftarrow E[i]$
15. $i \leftarrow i + 1$
16. **senão**
17. $V[k] \leftarrow D[j]$
18. $j \leftarrow j + 1$

O processo de seleção do menor valor e substituição do valor inicial de cada subvetor é repetido até que todos os valores sejam selecionados. Quando se atinge o valor ∞ , em um dos subvetores, a opção será sempre pela seleção dos valores no outro subvetor; veja que o valor selecionado é sempre o menor entre os subvetores e, desse modo, qualquer valor será sempre menor que ∞ .

A Figura 3 apresenta o passo a passo da execução do procedimento *Merge*() sobre um vetor de inteiros. O estado representado no passo (a) da Figura 3 considera os subvetores E e D já povoados com os valores do vetor original V . Desse modo, os ponteiros k , i e j identificam as posições iniciais do vetor V e dos subvetores E e D , respectivamente. As sentinelas, também, já estão consideradas nesse estado inicial, sendo atribuídas às posições finais de cada subvetor. Esse estado inicial é resultado das linhas 1-11, descritas no Algoritmo 5.

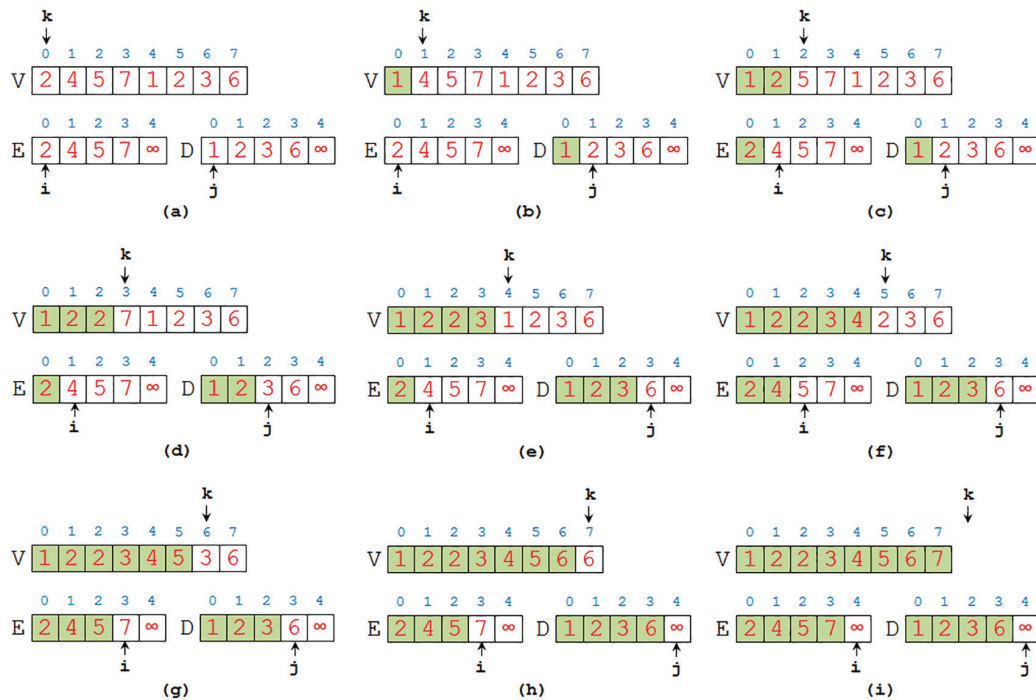


Figura 3 – Descrição da execução do procedimento *Merge* sobre um vetor de inteiros

Fonte: Acervo do conteudista

Os valores $E[i]$ e $D[j]$ são comparados para a seleção do menor entre eles. No exemplo, o menor valor, no estado representado pelo passo (a), é $D[0] = 1$. Assim, esse valor é atribuído à posição $k = 0$ no vetor V ; veja o resultado no passo (b), no qual os valores selecionado e atribuído são destacados na cor verde. Nesse estado atual, o vetor D tem como posição inicial $i = 1$, resultado da seleção anterior. Essa seleção é resultado da linha 12 do Algoritmo 5, que descreve um laço para a execução das comparações e seleção correspondente.

A próxima comparação será entre os valores em $E[0]$ e $D[1]$, em que ambas as posições possuem o mesmo valor; nesse caso, o valor selecionado será do subvetor E . Esse processo resulta no estado representado pelo passo (c), na Figura 3.

O processo de comparação e seleção dos menores valores se repete até que todo o vetor V tenha suas posições preenchidas, de forma ordenada, pelos valores intercalados dos subvetores E e D . O papel das sentinelas nos subvetores segue conforme descrito anteriormente, ou seja, marcam o final do respectivo subvetor.

O procedimento *Merge()* é funcional somente quando os subvetores estejam ordenados. Essa premissa não parece ser muito útil, visto que se espera que o algoritmo faça a ordenação, mas, nesse caso, a ordenação é um pré-requisito. Nesse contexto, precisamos de um procedimento complementar que resolva a questão da pré-ordenação.

Algoritmo 6 – Algoritmo de ordenação pelo método de intercalação (*Merge Sort*), esse procedimento é parte do algoritmo completo

O procedimento *Mergesort()*, descrito no Algoritmo 6, realiza a divisão do vetor de entrada V , de forma recursiva, até que cada subvetor tenha somente um elemento. Veja que, na linha 1 do Algoritmo 6, se os ponteiros p e r , que representam o início e o final

do vetor, respectivamente, foram iguais, isso significa que ambos apontam para a mesma posição, assim há somente uma posição no vetor. Nesse sentido, podemos afirmar que um vetor com somente um elemento já se encontra ordenado.

$MergeSort(V, p, r)$

1. **se** $p < r$
2. $q \leftarrow (p + r) / 2$
3. $MergeSort(V, p, q)$
4. $MergeSort(V, q + 1, r)$
5. $Merge(V, p, q, r)$

Podemos dizer que o algoritmo $MergeSort()$ desmembra o vetor em subvetores ordenados e com uma única posição. Assim, o procedimento $Merge()$ pode ser considerado sobre os subvetores, pois eles são ordenados. A recomposição do vetor a partir do procedimento $Merge()$ resultará em sua ordenação.

Considere a Figura 4, na qual há uma representação gráfica do desmembramento do vetor pelo procedimento $MergeSort()$ e a posterior recomposição por meio do procedimento $Merge()$, ambos descritos nos Algoritmos 6 e 5, respectivamente. O vetor inicial, que está representado na parte superior da Figura 4, tem os valores de suas posições desordenados. Veja que, a partir do procedimento $MergeSort()$, o vetor é decomposto em dois subvetores, sucessivas vezes, até que haja somente subvetores unitários. Essa decomposição é resultado das chamadas recursivas ao $MergeSort()$.

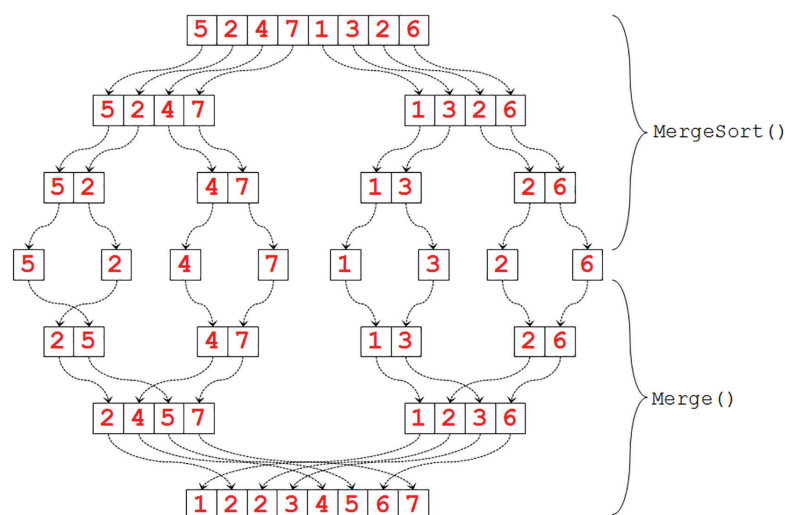


Figura 4 – Passo a passo da ordenação pelo método da intercalação

Fonte: Acervo do conteudista

A partir daí, cada instância unitária é recomposta, considerando os menores valores entre as posições iniciais de cada vetor, conforme discutido no procedimento $Merge()$. Cada chamada ao $Merge()$ recompõe um par de subvetores, inicialmente os unitários, depois os com dois elementos cada e, finalmente, os dois subvetores finais com quatro

elementos cada um. O estado final, após a execução do algoritmo, é a recomposição do vetor inicial, agora ordenado.

Assim como o algoritmo *Quick Sort*, o *Merge Sort* apresenta um desempenho melhor que os algoritmos de ordenação discutidos na unidade anterior. A estratégia da divisão e conquista, baseada na recursão, possibilita esse desempenho superior, em termos de tempo de execução.

Pesquisa Binária

Na unidade anterior tivemos a oportunidade de estudar a pesquisa sequencial. Vimos que esse tipo de pesquisa tem por objetivo verificar se um determinado elemento está ou não presente em um conjunto de dados - no nosso caso específico, em um vetor.

A pesquisa sequencial compara o elemento, para o qual queremos verificar a pertinência, com todos os elementos existentes no vetor de busca. Nesse contexto, a comparação é feita de forma sequencial com todos os elementos do vetor. Essa estratégia é simples, porém muito ineficiente. Se o elemento que buscamos estiver na última posição do vetor e supondo que esse vetor tenha mil elementos, teremos que realizar mil comparações. Quanto maior o vetor, maior é a possibilidade de realizar muitas comparações.

Há outra estratégia que podemos considerar para a busca em um vetor, ou outra estrutura de dados similar, que é denominada busca binária. A busca binária é realizada sobre um vetor previamente ordenado, essa é a premissa básica para que possamos melhorar a eficiência da nossa busca.

Antes de discutirmos os aspectos técnicos da busca binária, vamos iniciar com uma analogia para compreendermos a dinâmica da busca. Há muitos anos, quando era preciso encontrar o número do telefone de alguém, utilizava-se uma lista telefônica. Nessa lista, havia centenas de páginas com os nomes e números de telefones dos usuários cadastrados, em ordem alfabética.

A pesquisa por um nome na lista telefônica seguia os seguintes passos: primeiro, abria-se a lista aproximadamente ao meio e se verificava se o nome buscado estava na parte da esquerda ou da direita da lista. Veja que nesse ponto era possível descartar metade das páginas e a busca continuava, da mesma forma, na outra metade. O processo se repetia até que se encontrasse o nome pretendido.

A mesma lógica utilizada para se encontrar um nome em uma lista telefônica é a base da pesquisa ou busca binária. A premissa é que se realize a busca sobre um vetor previamente ordenado, de modo que, a cada passo, possamos desconsiderar metade do vetor. Esse procedimento reduz substancialmente o número de comparações que serão realizadas até que se encontre o valor procurado.

Algoritmo 7 – Algoritmo para busca binária

O Algoritmo 7 descreve as instruções que compõem o algoritmo de busca binária. Os parâmetros de entrada do *BuscaBinaria()* são: o valor a ser procurado (x), um vetor (V) previamente ordenado, as posições inicial (p) e final (r) do vetor.

O primeiro passo do *BuscaBinaria()* é a verificação do tamanho do vetor, considerando que a abordagem utilizada pelo algoritmo é a divisão e conquista, o vetor é dividido ao meio a cada nova chamada recursiva. Assim, quando uma chamada passar um valor para o marcador de início maior que o valor para o marcador de fim do vetor, isso significa que não há elementos a serem pesquisados. Nesse caso, o algoritmo não encontrou o valor procurado e ele retornará -1 para sinalizar essa situação.

```
BuscaBinaria( $x, V, p, r$ )  
1.  se  $p > r$   
2.      retornar  $-1$   
3.  senão  
4.       $q \leftarrow (p + r) / 2$   
5.      se  $V[q] = x$   
6.          retornar  $q$   
7.      senão  
8.          se  $V[q] < x$   
9.              retornar BuscaBinaria( $x, V, q + 1, r$ )  
10.         senão  
11.             retornar BuscaBinaria( $x, V, p, q - 1$ )
```

O algoritmo *BuscaBinaria()* calcula a posição média q do vetor, essa posição é utilizada para desmembrar o vetor em dois subvetores, um à direita da posição q e outro à esquerda. A partir daí existem três possibilidades. Caso o valor procurado esteja na posição q , essa posição é retornada como resposta (linhas 5-6). Por outro lado, se o valor na posição q for menor que o valor procurado, é feita uma chamada recursiva ao algoritmo *BuscaBinaria()*, passando como parâmetro o subvetor à direita de q . Finalmente, se o valor na posição q for maior que o valor procurado, a chamada recursiva passa o subvetor à esquerda de q como parâmetro.

O processo de busca se repetirá até que a posição que contém o valor procurado seja identificada e retornada como resposta. Caso a pesquisa não identifique o valor procurado, o algoritmo retornará o valor -1 , indicando que o valor não está presente no vetor.

A Figura 5 descreve o passo a passo para a execução do algoritmo *BuscaBinaria()* sobre um vetor de inteiros com 20 elementos ordenados, considerando o valor de busca $x = 23$. Veja que a condicional da linha 1 é satisfeita, pois temos $p = 0$ e $r = 19$, isso nos indica que existem elementos no vetor que devem ser pesquisados. O próximo passo é calcular a posição que desmembra o vetor em dois subvetores. A posição de desmembramento será $q = [(0 + 19) / 2] = 9$. Veja que o valor dessa posição ($V[9] = 22$) não é o valor procurado, sendo que esse valor é menor que o valor procurado. Assim, todo o subvetor esquerdo e a posição de desmembramento podem ser desconsiderados. Esse passo é representado na Figura 5a.

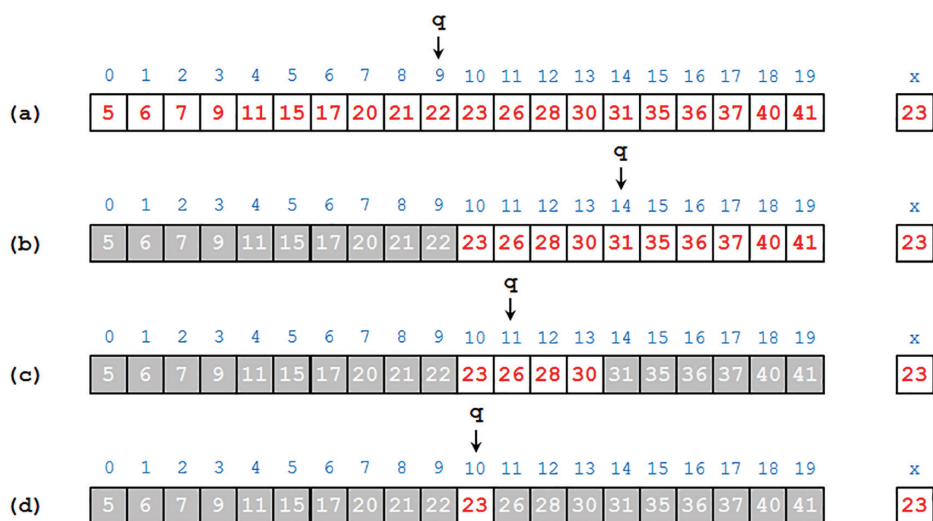


Figura 5 – Passo a passo da execução do algoritmo de busca binária

Fonte: Acervo do conteudista

O algoritmo fará uma chamada recursiva, enviando como parâmetro o subvetor direito ($V[10..19]$). O processo se repete com o cálculo da posição de desmembramento ($q = [(10 + 19) / 2] = 14$). Veja que, novamente, o valor da posição de desmembramento ($V[14] = 31$) não é igual ao valor procurado, sendo maior que o mesmo. Nesse caso, tanto a posição de desmembramento quanto o subvetor direito são desconsiderados. Esse passo pode ser observado na Figura 5b.

O terceiro passo, ilustrado na Figura 5c, considera o subvetor $V[10 \dots 13]$ e a posição de desmembramento é $q = \lfloor (10+13)/2 \rfloor = 11$. O valor procurado não corresponde ao valor da posição q , sendo menor que ele. Assim, ambos são desconsiderados, a posição q e o subvetor direito.

Finalmente, o subvetor esquerdo, remanescente do passo anterior, é enviado como parâmetro à chamada recursiva de *BuscaBinaria()*. Esse vetor apresenta um único elemento $V[10]$; desse modo, o cálculo da posição de desmembramento é $q = \lfloor (10 + 10)/2 \rfloor = 10$ e o valor dessa posição é $V[10] = 23$ o que corresponde ao valor procurado. O algoritmo, então, retorna a posição desse valor. Veja a Figura 5d.



Considere um vetor ordenado com 100 elementos, suponha que o elemento que buscamos está na última posição. Quantas comparações seriam necessárias para identificar o valor por meio da busca binária? E pela busca sequencial?

Nesta unidade tivemos a oportunidade de estudar os algoritmos de ordenação que consideram a abordagem da divisão e conquista por meio da recursividade. Esses algoritmos são muito eficientes, quando comparados àqueles estudados na unidade anterior. Além disso, vimos que a mesma técnica pode ser aplicada para pesquisa sobre um vetor ordenado. Essa aplicação, também, é muito mais eficiente que a busca sequencial, abordada anteriormente.

A recursividade é uma técnica útil, que deve ser considerada com moderação. Os algoritmos recursivos podem consumir uma grande quantidade de recursos, como, por exemplo, a memória.

Na próxima unidade, vamos retomar as pilhas e fila, a partir da alocação dinâmica de memória. Além disso, vamos ver como podemos estruturar listas de elementos, também, considerando alocação dinâmica de memória.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

🎥 Vídeos

Recursividade. Dicionário do Programador

<https://youtu.be/NKymAD4pJZI>

Quick-sort with Hungarian (Küküllőmenti legényes) folk dance

<https://youtu.be/ywWBy6J5gz8>

Merge Sort vs Quick Sort

<https://youtu.be/es2T6KY45cA>

Binary search with Flamenco dance

<https://youtu.be/iP897Z5Nerk>

Referências

ASCENCIO, A. F. G.; ARAÚJO, G. S. **Estruturas de dados**: algoritmos, análise da complexidade e implementações em *Java* e *C/C++*. São Paulo: Pearson, 2010. (*e-book*)

CORMEN, T. H. *et al.* **Introduction to algorithms**. MIT press, 2009.

SZWARCFITER, J. L. **Estruturas de dados e seus algoritmos**. 3ª ed. Rio de Janeiro: LTC, 2010. (*e-book*)



Cruzeiro do Sul
Educatonal